

Aristotle University of Thessaloniki

MSc Program in Biomedical Engineering

Course BM020- Machine Learning in biomedical data analysis 2025



Heart Failure Detection Technical Report

ΜΑΙΟΣ 2025

Professor: Ioanna Chouvarda

Students: Georgia Vetsika
Paraskevi Katsarela

Introduction

In our Heart Failure Detection project, we conducted a biomedical data analysis on a binary classification problem using traditional machine learning techniques, in order to increase the classifications accuracy. The techniques that are analyzed below are Random Forest (RF), Extreme Gradient Boosting (XGBOOST), Extreme Gradient Boosting with Hyperparameter Optimization (XGBOOST+ GridSearchCV), Support Vector Machine (SVM), Voting Classifier and lastly Feature Engineering on the best performing models of the above tests. Also, the above techniques were generated in multiple variations, in order to maximize the accuracy.

The original labels (0, 1, 2) were merged into a binary format, 0 (no heart failure) & 1 (heart failure). Then, the dataset was split into 80% training and 20% testing. We stratified the samples to ensure class balance, and no missing data were present.

The dataset includes information on heart condition detection features. The first feature is the Ejection Fraction (EF), which indicates how well the heart is pumping blood. The second feature is Global Longitudinal Strain (GLS), which reflects deformation of the heart muscle, a sensitive measure of cardiac dysfunction. The last feature is QRS complex taken from a electrocardiogram (ECG), which reflects electrical conduction based on the duration of the QRS signal.

Clinically, EF and GLS are both echocardiographic measures related to heart function. The ratio EF/GLS may capture some interaction or proportional relationship that could be more predictive than either feature alone. The resulting EF/GLS feature could reveal more about systolic function and help the model differentiate between patients with and without heart failure.

QRS is an electrical measurement, and GLS is a mechanical/functional measurement. The ratio could reflect electromechanical coupling or other latent relationships in cardiac behavior. This is a way to create interaction features that might uncover nonlinear patterns or dependencies between measurements. Lastly, it divides the QRS duration (from the ECG) by GLS.

A Random Forest Classifier is an ensemble learning method for classification, regression and other tasks that work by creating a multitude of decision trees during training. For classification tasks, the output of the random forest is the class selected by most trees. It is also an estimator that fits several decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting.

XGBOOST, which stands for "Extreme Gradient Boosting", is a machine learning model that is used for supervised learning problems, in which we use the training data to predict a target/response variable. It is designed for efficiency and scalability, making it significantly faster than traditional gradient boosting implementations. Furthermore, on the XGBOOST model we applied an optimization hyperparameter, XGBOOST+GridSearchCV. Instead of using manually chosen hyperparameters, GridSearchCV searches for the best combination by training many models and evaluating them via cross-validation. As a result, it significantly improves accuracy, and it reduces the biases (underfitting) and variances (overfitting).

Different configurations of SVM classifiers were explored for the binary classification task of detecting heart failure. A kernel is a method that changes input data into bigger space to help separate groups easily. SVM uses kernels to handle cases where data is not linearly separable in its original form. SVM kernel models are divided into linear and rbf (radial basis function) kernel models. Linear kernel models do not apply any transformation to the data into straight-line boundary to separate the classes. On the other hand, the RBF kernel maps data into an infinite-dimensional space, and it allows the SVM to draw complex curved boundaries.

Feature engineering is the process of selecting, manipulating and transforming raw data into features that can be used in supervised learning. In our application, feature engineering was applied in the context of predicting heart failure using clinical and ECG-based measurements.

Voting Classifier is an ensemble learning technique that combines multiple classifiers and predicts the class based on a voting mechanism. This approach enhances model accuracy, reduces overfitting, and makes predictions more robust. Also, it improves generalization and balances different model strengths.

There are two types of voting classifiers:

1. **Hard Voting:** The final prediction is based on the majority vote of individual classifiers. For instance, if three classifiers predict [Class A, Class B, Class A], the final prediction will be Class A.
2. **Soft Voting:** Instead of counting votes, it averages the predicted probabilities of each classifier and selects the class with the highest probability. For example: {[Class A, Class B, Class C]}/3.

In our case, the Voting Classifier combines all model evaluations, SVM, Random Forest & XGBoost. In this way, the biases are balanced, the predictions are stabilized, and the accuracy is improved on real time biomedical data (like heart failure detection).

Random Forest Model

The dataset that we have is perfectly balanced (class 0: 50%, class 1: 50%) and includes 120 samples.

HF	
0	0.5
1	0.5

Experiment 1: Base-line model

The first RF model that we tried was a base-line model with 100 trees (`n_estimators=100`) and no modifications to parameters.

```
# Train a Random Forest model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)
y_pred_train = model.predict(X_train)

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Random Forest Accuracy:", accuracy)

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))

from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=10) # 10-fold cross-validation
print("Cross-validated accuracy scores:", scores)
print("Mean CV accuracy:", scores.mean())
```

The results that we got are shown below.

```
Random Forest Accuracy: 0.9583333333333334
precision    recall  f1-score   support

      0       1.00      0.92      0.96         12
      1       0.92      1.00      0.96         12

 accuracy          0.96
 macro avg         0.96
weighted avg         0.96

Cross-validated accuracy scores: [1. 0.91666667 1. 0.91666667 0.83333333 0.91666667
 1. 1. 0.91666667 1.]
Mean CV accuracy: 0.95

Training Accuracy: 1.0000
Testing Accuracy: 0.9583
Training Error: 0.0000
Testing Error: 0.0417
Warning: Overfitting detected! Your model performs perfectly on the training data but not on the testing data.
```

Overall, a good random forest accuracy (~0,95). However:

- **Training accuracy**→ The model perfectly classifies all the training samples (1.0). This suggests that the model has probably memorized the training data, which leads to overfitting.
- **Testing accuracy**→ The fact that the testing accuracy, how the model performs in unseen data, is lower than the training accuracy, which is perfect, also suggests that the model is overfitting.
- **Cross-validated accuracy scores (CV)**→ We added this feature in order to ensure that the model truly performs well and not just by luck. In this process the data is split into 10 equal parts (folds). The 9 are used for training and the

remaining 1 is used for testing. This process is repeated 10 times, each time using a different fold as the test set and get back 10 accuracy scores. The final mean of these scores (**Mean CV accuracy**) gives a more reliable accuracy score. We can see that in our case, CV scores vary from 0.83 to 1.0 which shows instability in the predictions.

- **Precision**→ For the 0-class precision is 1.0, which means that the model **never wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and it caught **all HF cases**, which is excellent for safety.
- **F1-scores**→ **The same for both classes**. The model maintains a great balance of precision and recall.

Experiment 2: 'Max_features' parameter

In this trial, we added the parameter "max_features= ...". We tried 3 different cases where:

1. **max_features= 'log2'**→ Uses the base-2 logarithm of the number of features. We have 3 features so for each split the model takes into consideration 1 feature. The results we got back were identical as the ones using the baseline model.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Train a Random Forest model
model = RandomForestClassifier(n_estimators=100, max_features='log2', random_state=42)
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)
y_pred_train=model.predict(X_train)

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Random Forest Accuracy:", accuracy)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

2. **max_features= 2**→ Uses 2 out of the 3 features in every split. The results we got back were identical as the ones using the baseline model.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Train a Random Forest model
model = RandomForestClassifier(n_estimators=100, max_features= 2, random_state=42)
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)
y_pred_train=model.predict(X_train)

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Random Forest Accuracy:", accuracy)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

3. **max_features= None**→ Uses all features in every split. This also gave back the same results.

```
# Train a Random Forest model
model = RandomForestClassifier(n_estimators=100, max_features= None, random_state=42)
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)
y_pred_train=model.predict(X_train)

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Random Forest Accuracy:", accuracy)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

All 3 trials gave back the same exact results which indicates that due to the limited number of features, this parameter does not play a significant role on the overall performance of the model.

Experiment 3: 'Min_samples_split' parameter

In this part we added "min_samples_split=10". With this parameter, the complexity of the model is reduced, and each split must have at least 10 samples. The trees of the model grow less deep. The code and results are shown below.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Train a Random Forest model
model = RandomForestClassifier(n_estimators=100, max_features= 'log2', min_samples_split=10, random_state=42)
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)
y_pred_train=model.predict(X_train)

# Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Random Forest Accuracy:", accuracy)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
Random Forest Accuracy: 0.9583333333333334
precision    recall  f1-score   support

0           1.00     0.92     0.96         12
1           0.92     1.00     0.96         12

accuracy          0.96
macro avg         0.96     0.96     0.96         24
weighted avg      0.96     0.96     0.96         24

Cross-validated accuracy scores: [1. 0.91666667 1. 1. 0.91666667 0.91666667]
Mean CV accuracy: 0.9666666666666666
Training Accuracy: 0.9896
Testing Accuracy: 0.9583
Training Error: 0.0104
Testing Error: 0.0417
No obvious overfitting detected.
```

The results are improved. Despite that the **random forest accuracy** is the same as before, the **cross-validated accuracy scores** are improved (they are between 0.91-1.0 and the **mean cv accuracy** is ~0.96).

- **Training accuracy**→ The value has lowered to 0.9896 which is actually a good thing.
- **Testing accuracy**→ Has stayed the same as before. But now the difference between the training and testing accuracy is smaller.
- **Precision**→ For the 0-class precision is 1.0, which means that the model **never wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and it caught **all HF cases**, which is excellent for safety.

- **F1-scores**→ **The same for both classes.** The model maintains a great balance of precision and recall.

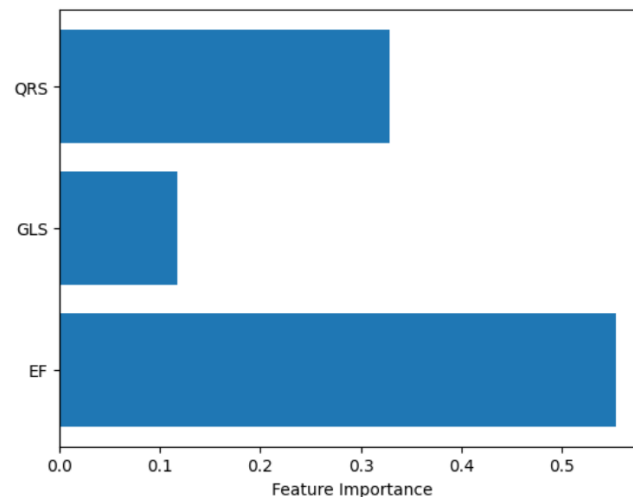
The model performs consistently across the folds and the overall accuracy has been improved. There is no sign of overfitting.

Experiment 4: GLS removal

We then decided to test how each feature contributes to the performance of the model keeping intact the other parameters that we used in Experiment 3.

```
import matplotlib.pyplot as plt

importances = model.feature_importances_
features = X.columns
plt.barh(features, importances)
plt.xlabel("Feature Importance")
plt.show()
```



Based on feature importance, **GLS** contributes the least, so the next step was to check if the model performs just as well without it.

```
Random Forest Accuracy: 0.9583333333333334
precision    recall  f1-score   support

0           1.00     0.92     0.96        12
1           0.92     1.00     0.96        12

accuracy          0.96     0.96     0.96        24
macro avg         0.96     0.96     0.96        24
weighted avg      0.96     0.96     0.96        24

Cross-validated accuracy scores: [1. 0.91666667 1. 1. 0.83333333 0.91666667]
Mean CV accuracy: 0.9583333333333333
```

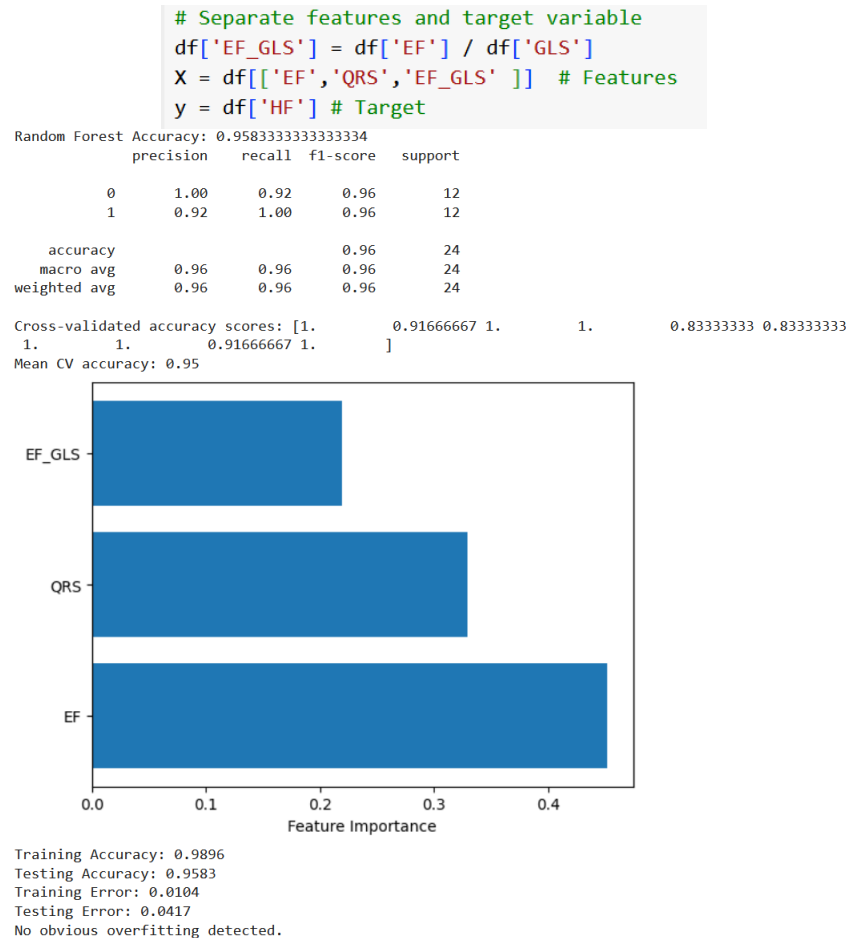
```
Training Accuracy: 0.9896
Testing Accuracy: 0.9583
Training Error: 0.0104
Testing Error: 0.0417
No obvious overfitting detected.
```

The model still performs well. We notice a drop in **Mean CV accuracy**, compared to the one in Experiment 3, which indicates that GLS has a role in the model but not a significant one. The results below are the same with the ones that are shown in the previous trial.

Experiment 5: Feature Engineering

In this experiment we tried different combinations to see what works best.

- **Replacing GLS with a new feature: EF/GLS**



This outcome results in lower Mean CV accuracy and higher variation of the cross-validated accuracy scores.

- **Precision**→ For the 0-class precision is 1.0, which means that the model **never wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and it caught **all HF cases**, which is excellent for safety.
- **F1-scores**→ **The same for both classes**. The model maintains a great balance of precision and recall.

- **Leaving GLS and adding a new feature: EF/GLS**

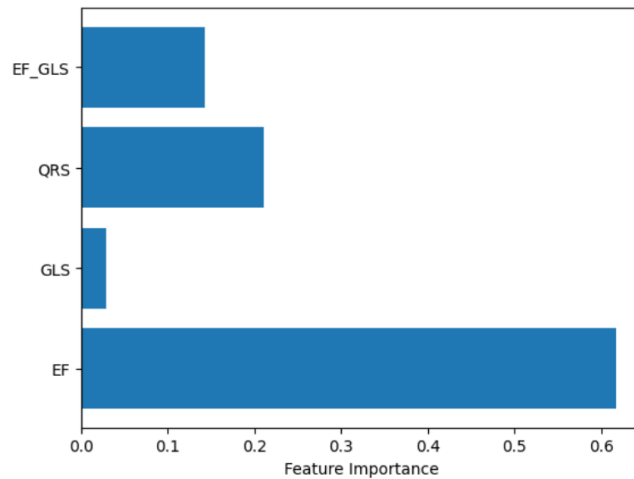
```
# Separate features and target variable
df['EF_GLS'] = df['EF'] / df['GLS']
X = df[['EF', 'GLS', 'QRS', 'EF_GLS']] # Features
y = df['HF'] # Target
```


Random Forest Accuracy: 0.9583333333333334

	precision	recall	f1-score	support
0	1.00	0.92	0.96	12
1	0.92	1.00	0.96	12
accuracy			0.96	24
macro avg	0.96	0.96	0.96	24
weighted avg	0.96	0.96	0.96	24

Cross-validated accuracy scores: [1. 0.91666667 1. 1. 0.83333333 0.83333333
1. 1. 0.91666667 1.]

Mean CV accuracy: 0.95



Training Accuracy: 0.9896
 Testing Accuracy: 0.9583
 Training Error: 0.0104
 Testing Error: 0.0417
 No obvious overfitting detected.

The results are exactly the same as the previous test. Which means that **EF/GLS** as a feature does not improve the model of Example 3.

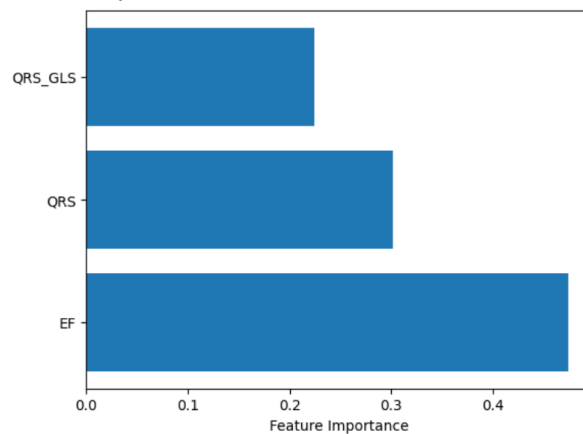
- **Removing GLS and adding a new feature: QRS/GLS**

Random Forest Accuracy: 0.9583333333333334

	precision	recall	f1-score	support
0	1.00	0.92	0.96	12
1	0.92	1.00	0.96	12
accuracy			0.96	24
macro avg	0.96	0.96	0.96	24
weighted avg	0.96	0.96	0.96	24

Cross-validated accuracy scores: [1. 0.91666667 1. 1. 0.83333333 0.91666667
1. 1. 1. 1.]

Mean CV accuracy: 0.9666666666666666

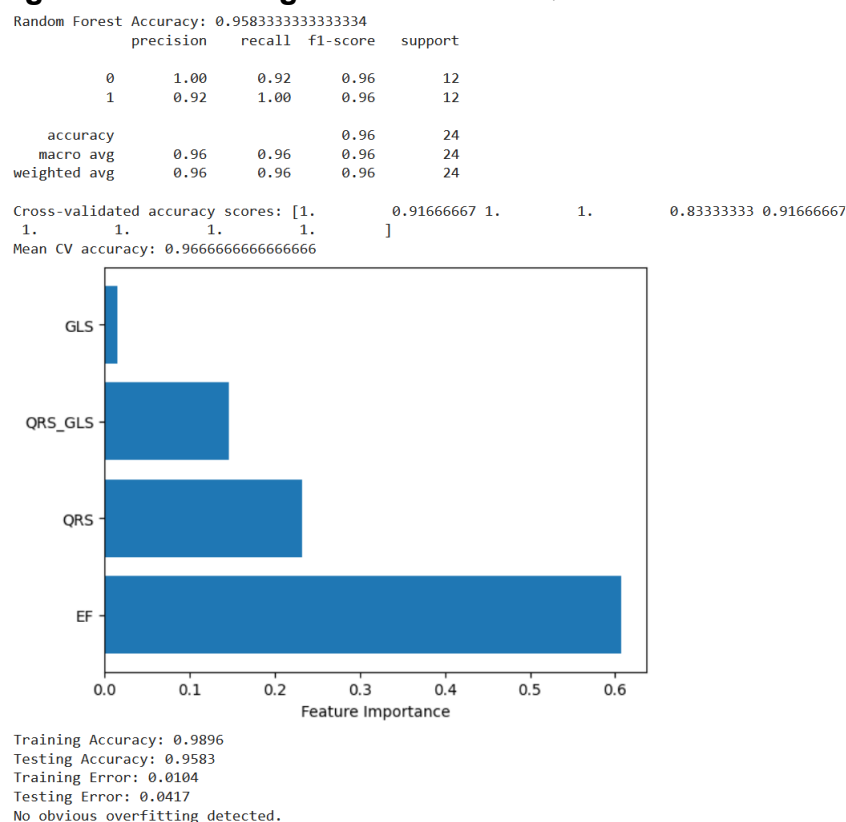


Training Accuracy: 0.9792
 Testing Accuracy: 0.9583
 Training Error: 0.0208
 Testing Error: 0.0417
 No obvious overfitting detected.

In this case we have the best **Mean CV accuracy** we had so far (equal with the one in Experiment 3). We notice a slight variance in the **cross-validated accuracy scores** (values from 0.83-1.0) and a slightly lower value of the **training accuracy** which could be acceptable if it is accompanied by good generalization performance.

- **Precision**→ For the 0-class precision is 1.0, which means that the model **never wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and it caught **all HF cases**, which is excellent for safety.
- **F1-scores**→ **The same for both classes**. The model maintains a great balance of precision and recall.

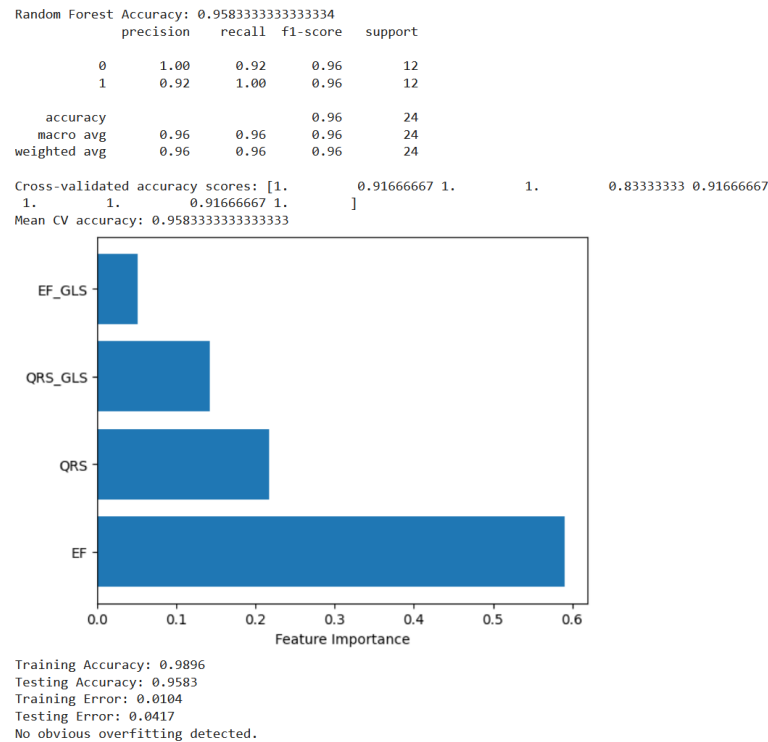
- **Leaving GLS and adding a new feature: QRS/GLS**



In this case we have again the best **Mean CV accuracy** we have had so far (equal with the one in Experiment 3). We notice also that we have the same results as before with a slightly higher level of **training accuracy**.

- **Precision**→ For the 0-class precision is 1.0, which means that the model **never wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and it caught **all HF cases**, which is excellent for safety.
- **F1-scores**→ **The same for both classes**. The model maintains a great balance of precision and recall.

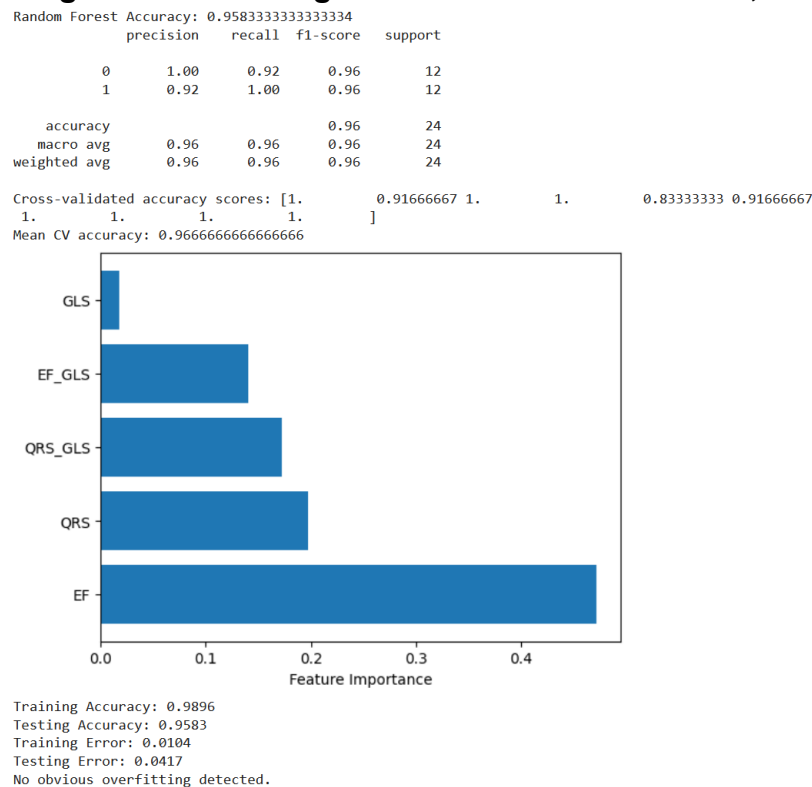
- **Removing GLS and adding both new features: EF/GLS, QRS/GLS**



In this case we have a lower **Mean CV accuracy** compared to the previous trial. We notice a slight variance in the **cross-validated accuracy scores** (values from 0.83-1.0) and the training accuracy remains the same as before.

- **Precision**→ For the 0-class precision is 1.0, which means that the model **never wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and it caught **all HF cases**, which is excellent for safety.
- **F1-scores**→ **The same for both classes**. The model maintains a great balance of precision and recall.

- **Leaving GLS and adding both new features: EF/GLS, QRS/GLS**



In this case we have again the best **Mean CV accuracy** we have had so far (equal with the one in Experiment 3). We notice a slight variance in the **cross-validated accuracy scores** (values from 0.83-1.0) compared to the ones in Experiment 3 and the other metrics are exactly the same.

Important note: After adding more features, we also tuned the 'max_features' parameter again (2, 3, None) and tried the different cases. These tests did not give back a **Mean accuracy CV** higher than 0.96.

Outcome

- Reviewing all our Random Forest models we concluded that the three best ones are: **Experiment 3, Trial of leaving GLS and adding both new features: EF/GLS, QRS/GLS, Trial of Leaving GLS and adding a new feature: QRS/GLS.**
- Comparing these three models, that in general have similar performance, we conclude that the results of **Experiment 3** are better because the cross-validation scores have fewer variations (0.91-1.0) and the **recall** metric for HF is 100%.

XGBOOST / XGBOOST GRID SEARCH CV

Experiment 1

```
# Initialize and train XGBoost model
xgb_model = XGBClassifier(eval_metric='logloss')
xgb_model.fit(X_train, y_train)
```

We are using the default XGBOOST model which contains the following parameters:

Parameter	Default Value	Description
n_estimators	100	Number of trees
max_depth	6	Maximum depth of each tree
learning_rate	0.3	Step size
subsample	1.0	Fraction of training samples used per tree
colsample_bytree	1.0	Fraction of features used per tree
objective	'binary:logistic'	Used for binary classification tasks
booster	'gbtree'	Type of boosting model
randm_state	None	No fixed seed unless specified

Table 1. XGBOOST parameters.

```
Training Accuracy: 0.9896
Testing Accuracy: 0.9167
Training Error: 0.0104
Testing Error: 0.0833

Classification Report (Test Data):
              precision    recall  f1-score   support

     0       0.92        0.92        0.92        12
     1       0.92        0.92        0.92        12

 accuracy          0.92
 macro avg         0.92
 weighted avg      0.92

Cross-validated accuracy scores: [0.95833333 1.         0.875         0.95833333 0.95833333]
Mean CV accuracy: 0.95

No obvious overfitting detected.
```

The model is performing well. We notice high **training accuracy** (but not 1.0, which is good, probably it's learning well without memorizing). **Testing accuracy** is also good (91.67%) but it has a difference from the training accuracy, which might indicate serious overfitting.

- **Precision**→ For the 0 -class precision is 0.92, which means that the model **wrongly labelled** someone as "no HF" when they had HF. Of those predicted as "has HF", **92% had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and 92% of **HF cases**, which is not good for safety.

- **F1-scores**→ **The same for both classes.** The model maintains a good balance of precision and recall.

We also notice that **cross-validation scores** have a bit of a variance (0.87-1.0).

Experiment 2

We believe that the overall performance could be improved, so we applied in our code a **GridSearchCV** command that tries different types of hyperparameters (max_depth, learning_rate), evaluates each type by using cross-validation and returns the best-performing combination.

```
Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50, 'subsample': 0.8}
Best Cross-Validated Accuracy: 0.9579

Training Accuracy: 0.9688
Testing Accuracy: 0.9167
Training Error: 0.0312
Testing Error: 0.0833

Classification Report (Test Data):
      precision    recall  f1-score   support

     0       0.92       0.92       0.92        12
     1       0.92       0.92       0.92        12

 accuracy          0.92       0.92       0.92        24
 macro avg          0.92       0.92       0.92        24
weighted avg          0.92       0.92       0.92        24

Cross-validated accuracy scores: [0.95833333 1.          0.83333333 1.          0.95833333]
Mean CV accuracy: 0.95

No obvious overfitting detected.
```

By observing the results and comparing them with before, the **test accuracy** remained the same but the **training accuracy** slightly dropped, this is a good sign that the model is now less overfitted. The model uses fewer trees (n_estimators=50) and a shallower tree depth (max_depth=3) with low learning rate (0.01), making the model simpler and more robust. **Cross-validation accuracy** improved slightly, showing that the model is now more stable across different splits of the data.

- **Precision**→ For the 0 -class precision is 0.92, which means that the model **wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and 92% of **HF cases**, which is not good for safety.
- **F1-scores**→ **The same for both classes.** The model maintains a good balance of precision and recall.

Experiment 3: GLS REMOVAL

```
Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 100, 'subsample': 0.8}
Best Cross-Validated Accuracy: 0.9684

Training Accuracy: 0.9792
Testing Accuracy: 0.9503
Training Error: 0.0208
Testing Error: 0.0417

Classification Report (Test Data):
      precision    recall  f1-score   support

     0       1.00       0.92       0.96        12
     1       0.92       1.00       0.96        12

 accuracy          0.96       0.96       0.96        24
 macro avg          0.96       0.96       0.96        24
weighted avg          0.96       0.96       0.96        24

Cross-validated accuracy scores: [0.95833333 1.          0.875      1.          1.          ]
Mean CV accuracy: 0.9666666666666668

No obvious overfitting detected.
```

The results are improved compared to the model of Experiment 2. The **Mean CV accuracy** is higher and the **cross-validated accuracy scores** are improved (we have fewer variations).

- **Training accuracy**→ The value has increased to 0.9792, compared to the XGBOOST GRID model.
- **Testing accuracy**→ Has the most significant change. It improved from 0.9167 to 0.9583. Now testing and training accuracy are even more close together indicating that the model performs better without the GLS feature.
- **Precision**→ For the 0 -class precision is 1.0, which means that the model hasn't **wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and 100% of **HF cases**, which is excellent for safety.
- **F1-scores**→ **The same for both classes and higher than previously by 0.04**. The model maintains a very good balance of precision and recall.

Experiment 4: Feature Engineering

All the changes were done to the tuned (XGBOOST GRIDSEARCH CV) model because it had a better performance than the original XGBOOST.

- **Replacing GLS with a new feature: EF/GLS**

```
Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50, 'subsample': 0.8}
Best Cross-Validated Accuracy: 0.9579
```

```
Training Accuracy: 0.9688
Testing Accuracy: 0.9167
Training Error: 0.0312
Testing Error: 0.0833
```

```
Classification Report (Test Data):
      precision    recall  f1-score   support

     0       0.92      0.92      0.92        12
     1       0.92      0.92      0.92        12

 accuracy      0.92      0.92      0.92        24
 macro avg     0.92      0.92      0.92        24
 weighted avg   0.92      0.92      0.92        24
```

```
Cross-validated accuracy scores: [0.95833333 1.         0.875      1.         0.95833333]
Mean CV accuracy: 0.9583333333333334
```

```
No obvious overfitting detected.
```

By observing the results, we notice that they are exactly the same with the results of the XGBOOST GRID model, which means that removing GLS and then adding the feature EF/GLS is the same with keeping GLS in the model. This feature has no positive impact in our model.

- **Leaving GLS and adding a new feature: EF/GLS**

Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50, 'subsample': 0.8}
 Best Cross-Validated Accuracy: 0.9579

Training Accuracy: 0.9688
 Testing Accuracy: 0.9167
 Training Error: 0.0312
 Testing Error: 0.0833

Classification Report (Test Data):				
	precision	recall	f1-score	support
0	0.92	0.92	0.92	12
1	0.92	0.92	0.92	12
accuracy			0.92	24
macro avg	0.92	0.92	0.92	24
weighted avg	0.92	0.92	0.92	24

Cross-validated accuracy scores: [0.91666667 1. 0.83333333 1. 0.95833333]
 Mean CV accuracy: 0.9416666666666667

No obvious overfitting detected.

In this case the results have slightly changed. The **Training** and **Testing accuracy** have stayed the same, compared to the previous trial and the original XGBOOST GRID model, but the **Mean CV accuracy** has dropped. It is evident that the **cross-validated scores** also dropped which indicates that the model is no longer performing so well as previously with this change. All the other metrics are the same.

- **Replacing GLS with a new feature: QRS/GLS**

Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50, 'subsample': 0.8}
 Best Cross-Validated Accuracy: 0.9579

Training Accuracy: 0.9896
 Testing Accuracy: 0.9167
 Training Error: 0.0104
 Testing Error: 0.0833

Classification Report (Test Data):				
	precision	recall	f1-score	support
0	1.00	0.83	0.91	12
1	0.86	1.00	0.92	12
accuracy			0.92	24
macro avg	0.93	0.92	0.92	24
weighted avg	0.93	0.92	0.92	24

Cross-validated accuracy scores: [0.95833333 1. 0.83333333 0.95833333 0.95833333]
 Mean CV accuracy: 0.9416666666666668

No obvious overfitting detected.

In this trial the outcome differs more. The **Training** and **Testing accuracy** remained the same. The **precision** for 0 has improved to 1.0 and for 1 has dropped to 0.86, which indicates that the model mislabeled patients. We have higher **recall rates** for HF (better safety) and a drop in the **recall rate** of no HF, this shows that the model **caught all sick patients** but might flag a few healthy ones too. We also notice a drop in **f1-score** for no HF. The **cross-validated accuracy** also seems to have dropped and the **Mean CV accuracy** is slightly lower.

- **Leaving GLS and adding a new feature: QRS/GLS**

```
Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50, 'subsample': 0.8}
Best Cross-Validated Accuracy: 0.9579

Training Accuracy: 0.9688
Testing Accuracy: 0.9167
Training Error: 0.0312
Testing Error: 0.0833

Classification Report (Test Data):
      precision    recall  f1-score   support

     0       0.92       0.92       0.92        12
     1       0.92       0.92       0.92        12

 accuracy       0.92       0.92       0.92        24
 macro avg       0.92       0.92       0.92        24
weighted avg       0.92       0.92       0.92        24

Cross-validated accuracy scores: [0.91666667 1.          0.83333333 0.95833333 0.95833333]
Mean CV accuracy: 0.9333333333333333

No obvious overfitting detected.
```

In this trial the outcome is mostly the same as the original XGBOOST GRID with some minor changes. The **Training accuracy** and **Testing accuracy** remained the same compared to the model of Experiment 3. The **cross-validated accuracy scores** also seem to have dropped, and the **Mean CV accuracy** is lower.

- **Precision**→ For the 0 -class precision is 0.92, which means that the model **wrongly labelled** someone as "no HF" when they actually had HF. Of those predicted as "has HF", **92% actually had it**.
- **Recall**→ The model correctly identified **92% of the actual no-HF patients** and 92% of **HF cases**, which is not good for safety.
- **F1-scores**→ **The same for both classes**. The model maintains a good balance of precision and recall.

- **Removing GLS and adding two new features: EF/GLS, QRS/GLS**

```
Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50, 'subsample': 0.8}
Best Cross-Validated Accuracy: 0.9579

Training Accuracy: 0.9688
Testing Accuracy: 0.9167
Training Error: 0.0312
Testing Error: 0.0833

Classification Report (Test Data):
      precision    recall  f1-score   support

     0       0.92       0.92       0.92        12
     1       0.92       0.92       0.92        12

 accuracy       0.92       0.92       0.92        24
 macro avg       0.92       0.92       0.92        24
weighted avg       0.92       0.92       0.92        24

Cross-validated accuracy scores: [0.91666667 1.          0.83333333 0.95833333 0.95833333]
Mean CV accuracy: 0.9333333333333333

No obvious overfitting detected.
```

In this case, the results are the same as the previous trial. We believe, based on the tests we have done up until now, that GLS and EF/GLS have the same effect on the model.

- **Leaving GLS and adding two new features: EF/GLS, QRS/GLS**

```
Best Parameters: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 50, 'subsample': 0.8}
Best Cross-Validated Accuracy: 0.9579

Training Accuracy: 0.9688
Testing Accuracy: 0.9167
Training Error: 0.0312
Testing Error: 0.0833

Classification Report (Test Data):
      precision    recall  f1-score   support

     0       0.92      0.92      0.92        12
     1       0.92      0.92      0.92        12

 accuracy          0.92      0.92      0.92        24
 macro avg          0.92      0.92      0.92        24
 weighted avg          0.92      0.92      0.92        24

Cross-validated accuracy scores: [0.91666667 1.          0.83333333 1.          0.95833333]
Mean CV accuracy: 0.9416666666666667

No obvious overfitting detected.
```

All the results until the **cross-validated accuracy scores** are the same as previously. The **cross-validated accuracy scores** seem to have been slightly improved and this has caused the overall **Mean CV accuracy** to have risen

Outcome

- Reviewing all our XGBOOST models we concluded that the best one is: **XGBOOST GRIDSEARCH CV with GLS removal.**
- Comparing the models, we conclude that the results of the chosen trial are better because the cross-validation scores have fewer variations (0.91-1.0), the overall **Mean CV accuracy** is higher and the **recall** metric for HF is 100% which is crucial for safety reasons.

Support Vector Machine (SVM) - Model Evaluation and Comparison

We explored different configurations of SVM classifiers for the binary classification task of detecting heart failure. The dataset consisted of 120 samples with three features (EF, GLS, QRS) and a binary label indicating heart failure (1 for failure, 0 for no failure). Below is the comparative analysis of each SVM configuration.

In SVM, parameter C is a regularization parameter that controls the trade-off between achieving a low training error and a low testing error (generalization). Also, it controls how much the SVM tries to avoid misclassifying each training example. In our cases we tested C=1 & C=10, where in C=1 the SVM will try to keep the margin wide, but will also allow a few misclassifications if needed for generalization, and in C=10 the SVM will penalize misclassifications much more, aiming to classify the training data very well, possibly at the cost of overfitting.

Model 1: Linear SVM (C=1)

The first SVM model is a linear model with regularization parameter equal to 1 (C=1).

```
# Separate features and target variable
X = df[['EF', 'GLS', 'QRS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='linear', C=1)
svm_model.fit(X_train, y_train)

# Predict
y_pred_svm = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)
```

The results that we got are shown below.

```

SVM Accuracy: 1.0
              precision    recall  f1-score   support

         0         1.00      1.00      1.00        12
         1         1.00      1.00      1.00        12

 accuracy          1.00
  macro avg         1.00      1.00      1.00        24
 weighted avg       1.00      1.00      1.00        24

Cross-validated accuracy scores: [1.          0.91666667 0.91666667 0.91666667 0.95833333]
Mean CV accuracy: 0.9416666666666667
Training Accuracy: 0.9688
Testing Accuracy: 1.0000
Training Error: 0.0312
Testing Error: 0.0000
No obvious overfitting detected.

```

Model 1 demonstrated and retained testing accuracy (1.0) alongside strong generalization as demonstrated in cross-validation. The training errors are small (0.0312), and the margin is not too large, which indicates a good fit.

Model 2: Linear SVM (C=10)

The second SVM model is a linear model with regularization parameter equal to 10 (C=10).

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='linear', C=10)
svm_model.fit(X_train, y_train)

# Predict
y_pred_svm = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)

from sklearn.metrics import accuracy_score

# Evaluate
accuracy_svm = accuracy_score(y_test, y_pred_svm)
print("SVM Accuracy:", accuracy_svm)

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_svm))

```

```

SVM Accuracy: 1.0
              precision    recall  f1-score   support

         0         1.00      1.00      1.00        12
         1         1.00      1.00      1.00        12

 accuracy          1.00
  macro avg         1.00      1.00      1.00        24
 weighted avg       1.00      1.00      1.00        24

Cross-validated accuracy scores: [1.          0.91666667 0.91666667 0.91666667 0.95833333]
Mean CV accuracy: 0.9416666666666667
Training Accuracy: 0.9688
Testing Accuracy: 1.0000
Training Error: 0.0312
Testing Error: 0.0000
No obvious overfitting detected.

```

Model 2 was generated with increased regularization strength (C), that did not yield performance improvements. The results are identical to the previous configuration, showing that the linear model is already optimal within this range. On the other hand, in both cases SVM cross validation accuracy is high (~0.95).

Model 3: RBF Kernel SVM (C=1)

The first rbf kernel SVM model uses regularization parameter equal to 1 (C=1).

```
# Separate features and target variable
x = df[['EF', 'QRS', 'GLS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=1)
svm_model.fit(X_train, y_train)

# Predict
y_pred_svm = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)
```

```
SVM Accuracy: 1.0
precision    recall  f1-score   support

      0      1.00      1.00      1.00        12
      1      1.00      1.00      1.00        12

 accuracy          1.00          1.00          1.00          24
  macro avg          1.00          1.00          1.00          24
weighted avg          1.00          1.00          1.00          24

Cross-validated accuracy scores: [0.91666667 0.91666667 0.875      0.95833333 1.        ]
Mean CV accuracy: 0.9333333333333332
Training Accuracy: 0.9271
Testing Accuracy: 1.0000
Training Error: 0.0729
Testing Error: 0.0000
No obvious overfitting detected.
```

The RBF kernel Model 3 with reduced regularization allowed more flexibility in the margin, leading to slightly lower training performance (0.9271) and cross-validation accuracy (0.9333). Despite this, it still achieved perfect test set classification, though with a hint that the model is more prone to underfitting on training data compared to others.

Model 4: RBF Kernel SVM (C=10)

The second rbf kernel SVM model uses regularization parameter equal to 10 (C=10).

```

# Separate features and target variable
X = df[['EF', 'QRS', 'GLS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=10)
svm_model.fit(X_train, y_train)

# Predict
y_pred_svm = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)

```

```

SVM Accuracy: 1.0
      precision    recall  f1-score   support

      0         1.00      1.00      1.00        12
      1         1.00      1.00      1.00        12

   accuracy          1.00
  macro avg          1.00
weighted avg          1.00

Cross-validated accuracy scores: [1.          0.91666667 0.875      1.          1.          ]
Mean CV accuracy: 0.9583333333333333
Training Accuracy: 0.9479
Testing Accuracy: 1.0000
Training Error: 0.0521
Testing Error: 0.0000
No obvious overfitting detected.

```

The RBF kernel Model 4 introduces non-linearity, and here it slightly decreased the training accuracy (0.9479) but increased the average cross-validation score (0.9583). This suggests that the non-linear decision boundary better generalizes across the folds and may capture subtle patterns that a linear boundary does not.

Comparison of all SVM Models

All SVM models achieved high accuracy due to the balanced and clean nature of the dataset. However, SVM with RBF kernel and C=10 with 95.83% provided the best cross-validation performance, suggesting it may generalize slightly better under variation. On the other hand, Linear SVM and C=1 is the simplest and most efficient model without reducing its performance. Lastly, all configurations achieved 100% test accuracy, suggesting high separability of classes in feature space, possibly due to the informative nature of features EF, GLS, and QRS.

SVM Models Without GLS Feature - Model Evaluation and Comparison

In the below model evaluation and comparison, we excluded the GLS parameter, to retrain and evaluate four different SVM configurations, two with a linear kernel and two with a radial basis function (RBF) kernel, each using C=1 and C=10. The purpose was to assess the importance of GLS and test the generalizability of simplified models.

Firstly, we merged multi-class HF labels into binary: 0 (no heart failure), 1 (heart failure). Then only EF and QRS input features were used. The data were stratified to preserve label distribution in both train and test sets (50/50 class balance). Overall, no missing data were detected.

Model 5: Linear SVM without GLS (C=1)

The first linear kernel SVM model uses regularization parameter equal to 1 (C=1), with excluded GLS feature.

```
# Separate features and target variable
X = df[['EF', 'QRS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='linear', C=1)
svm_model.fit(X_train, y_train)

# Predict
y_pred_svm = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)
```

	precision	recall	f1-score	support
0	1.00	0.92	0.96	12
1	0.92	1.00	0.96	12
accuracy			0.96	24
macro avg	0.96	0.96	0.96	24
weighted avg	0.96	0.96	0.96	24

Cross-validated accuracy scores: [0.95833333 0.91666667 0.875 1. 0.95833333]
Mean CV accuracy: 0.9416666666666667
Training Accuracy: 0.9688
Testing Accuracy: 0.9583
Training Error: 0.0312
Testing Error: 0.0417
No obvious overfitting detected.

Model 5 linear SVM performed with a few misclassifications. Nonetheless, the overall performance is strong and comparable (CV=0.9416), suggesting linear separation is nearly sufficient even without GLS.

Model 6: Linear SVM without GLS (C=10)

The second linear kernel SVM model uses regularization parameter equal to 10(C=10), with excluded GLS feature.

```
# Separate features and target variable
X = df[['EF', 'QRS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='linear', C=10)
svm_model.fit(X_train, y_train)

# Predict
y_pred_svm = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)
```

```

SVM Accuracy: 0.9583333333333334
precision    recall  f1-score   support

      0       1.00      0.92      0.96        12
      1       0.92      1.00      0.96        12

 accuracy          0.96        0.96      0.96        24
 macro avg          0.96        0.96      0.96        24
 weighted avg          0.96        0.96      0.96        24

Cross-validated accuracy scores: [0.95833333 0.91666667 0.875      1.          0.95833333]
Mean CV accuracy: 0.9416666666666667
Training Accuracy: 0.9688
Testing Accuracy: 0.9583
Training Error: 0.0312
Testing Error: 0.0417
No obvious overfitting detected.

```

Model 6 with increased C did not lead to improvement in either training or testing results. The consistency indicates a robust and well-separated feature space even with only EF and QRS.

Model 7: RBF SVM without GLS (C=1)

The first rbf kernel SVM model uses regularization parameter equal to 1 (C=1), with excluded GLS feature.

```

# Select features (exclude GLS)
X = df[['EF', 'QRS']]
y = df['HF']

# Print class distribution
print("HF class distribution:\n", y.value_counts(normalize=True))

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

# Train SVM
svm_model = SVC(kernel='rbf', C=1)
svm_model.fit(X_train, y_train)

# Predictions
y_pred_test = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)

```

```

Training Accuracy: 0.9479
Testing Accuracy: 1.0000
Training Error: 0.0521
Testing Error: 0.0000

Classification Report:
precision    recall  f1-score   support

      0       1.00      1.00      1.00        12
      1       1.00      1.00      1.00        12

 accuracy          1.00        1.00      1.00        24
 macro avg          1.00        1.00      1.00        24
 weighted avg          1.00        1.00      1.00        24

Cross-validated accuracy scores: [1.          0.91666667 0.875      1.          1.          ]
Mean CV accuracy: 0.9583333333333333
No obvious overfitting detected.

```

Model 7 RBF kernel with lower regularization shows excellent generalization, achieving perfect test accuracy (1.0). The slightly lower training accuracy (0.9479) indicates that the model doesn't overfit, even in the absence of GLS.

Model 8: RBF SVM without GLS (C=10)

The second rbf kernel SVM model uses regularization parameter equal to 10 (C=10), with excluded GLS feature.


```

# Select features (exclude GLS)
X = df[['EF', 'QRS']]
y = df['HF']

# Print class distribution
print("HF class distribution:\n", y.value_counts(normalize=True))

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

# Train SVM
svm_model = SVC(kernel='rbf', C=10)
svm_model.fit(X_train, y_train)

# Predictions
y_pred_test = svm_model.predict(X_test)
y_pred_train = svm_model.predict(X_train)

```

```

Training Accuracy: 0.9479
Testing Accuracy: 1.0000
Training Error: 0.0521
Testing Error: 0.0000

Classification Report:
              precision    recall  f1-score   support

     0       1.00        1.00        1.00        12
     1       1.00        1.00        1.00        12

 accuracy          1.00
 macro avg          1.00
 weighted avg       1.00

Cross-validated accuracy scores: [0.95833333 0.91666667 0.875      1.         1.         ]
Mean CV accuracy: 0.95

No obvious overfitting detected.

```

Model 8 with raised regularization strength had no impact on training or test performance but slightly reduced mean cross-validation accuracy (0.95). This suggests that increasing C past 1 provides diminishing returns in this configuration.

Comparison of all SVM Models without GLS feature

Even after removing GLS, the classification performance remained high across all SVM models. RBF models SVM without GLS with C=1 proved to be the most effective, offering both perfect test set performance and the highest cross-validation accuracy. This reinforces the conclusion that EF and QRS alone are sufficiently informative for heart failure classification in this dataset. In conclusion, GLS exclusion had a minimal impact on accuracy, although all models had high performance. Overall, linear SVM proved to be viable, offering simplicity particularly when model interpretability and training speed are priorities.

Below is a comprehensive comparison table of SVM model performance with and without the GLS feature, including both linear and RBF kernels with different regularization strengths (C = 1 and C = 10):

SVM Model Comparison: With vs. Without GLS

Kernel	C	Features Used	Train Accuracy	Test Accuracy	CV Accuracy	Overfitting
Linear	1	EF, GLS, QRS	96.88%	100%	94.17%	No
Linear	10	EF, GLS, QRS	96.88%	100%	94.17%	No

RBF	1	EF, GLS, QRS	92.71%	100%	93.33%	Likely
RBF	10	EF, GLS, QRS	94.79%	100%	95.83%	No
Linear	1	EF, QRS	96.88%	95.83%	94.17%	No
Linear	10	EF, QRS	96.88%	95.83%	94.17%	No
RBF	1	EF, QRS	94.79%	100%	95.83%	No
RBF	10	EF, QRS	94.79%	100%	95.0%	No

Table 2. Comparison of all SVM models (with & without GLS).

Conclusion for all SVM Models

First and foremost, as listed in Table 1, it proves that removing GLS did not degrade test accuracy in most cases, especially with RBF kernels, which still achieved 100% test accuracy.

The best performance mode proved to be RBF Kernel, with perfect generalization in both feature sets (C=1, C=10).

Comparing Cross-Validation scores, it is observed a slight drop in CV accuracy without GLS approximately 1–2%, but still strong above 94%.

Regarding overfitting risk, RBF models with GLS & C=1 showed signs of overfitting, perfect training & test accuracy but lower CV. On the other hand, RBF models with C=1 and RBF models without GLS generalize better, reducing overfitting concerns.

Based on test accuracy, cross-validation score, and overfitting risk, the best overall model is SVM with RBF kernel C = 10 (Model 4) & SVM with RBF kernel without GLS & C = 1 (Model 7), which had same results as outcome.

Feature Engineering in SVM

The best predictive model turned out to be SVM with a radial basis function (RBF) kernel and regularization parameter $C=10$, chosen for its superior accuracy in earlier evaluations (CV accuracy=95.83%). The test was also conducted to the second model that had the same results (SVM rbf Kernel without GLS & $C=1$), but the accuracy outcome of the tests were lower, approximately 92%, and only the results from SVM rbf Kernel $C=10$ are presented below.

6 different combinations of the features (EF, GLS, QRS) were conducted to test how each set contributes to classification performance.

Experiment 1: EF_GLS

```
# Separate features and target variable
df['EF_GLS']=df['EF']/df['GLS']
X = df[['EF', 'QRS', 'GLS', 'EF_GLS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=10)
svm_model.fit(X_train, y_train)
```

```
SVM Accuracy: 1.0
      precision    recall  f1-score   support

      0         1.00      1.00      1.00        12
      1         1.00      1.00      1.00        12

   accuracy         1.00
  macro avg         1.00
 weighted avg         1.00

Cross-validated accuracy scores: [1.          0.91666667 0.875      1.          1.          ]
Mean CV accuracy: 0.9583333333333333
Training Accuracy: 0.9479
Testing Accuracy: 1.0000
Training Error: 0.0521
Testing Error: 0.0000
No obvious overfitting detected.
```

Experiment 2: EF_GLS, Without GLS feature

```
# Separate features and target variable
df['EF_GLS']=df['EF']/df['GLS']
X = df[['EF', 'QRS', 'EF_GLS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=10)
svm_model.fit(X_train, y_train)
```

```

SVM Accuracy: 1.0
precision    recall  f1-score   support

      0       1.00      1.00      1.00        12
      1       1.00      1.00      1.00        12

 accuracy
macro avg       1.00      1.00      1.00        24
weighted avg     1.00      1.00      1.00        24

Cross-validated accuracy scores: [1.          0.91666667  0.875         1.          1.          ]
Mean CV accuracy: 0.9583333333333333
Training Accuracy: 0.9479
Testing Accuracy: 1.0000
Training Error: 0.0521
Testing Error: 0.0000
No obvious overfitting detected.

```

Experiment 3: QRS_GLS

```

# Separate features and target variable
df['QRS_GLS']=df['QRS']/df['GLS']
X = df[['EF','QRS','GLS','QRS_GLS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=10)
svm_model.fit(X_train, y_train)

```

```

SVM Accuracy: 1.0
precision    recall  f1-score   support

      0       1.00      1.00      1.00        12
      1       1.00      1.00      1.00        12

 accuracy
macro avg       1.00      1.00      1.00        24
weighted avg     1.00      1.00      1.00        24

Cross-validated accuracy scores: [1.          0.91666667  0.875         1.          1.          ]
Mean CV accuracy: 0.9583333333333333
Training Accuracy: 0.9479
Testing Accuracy: 1.0000
Training Error: 0.0521
Testing Error: 0.0000
No obvious overfitting detected.

```

Experiment 4: QRS_GLS, Without GLS feature

```

# Separate features and target variable
df['QRS_GLS']=df['QRS']/df['GLS']
X = df[['EF','QRS','QRS_GLS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=10)
svm_model.fit(X_train, y_train)

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	12
1	1.00	1.00	1.00	12
accuracy			1.00	24
macro avg	1.00	1.00	1.00	24
weighted avg	1.00	1.00	1.00	24

Cross-validated accuracy scores: [1. 0.91666667 0.875 1. 1.]
 Mean CV accuracy: 0.9583333333333333
 Training Accuracy: 0.9479
 Testing Accuracy: 1.0000
 Training Error: 0.0521
 Testing Error: 0.0000
 No obvious overfitting detected.

Experiment 5: EF_GLS & QRD_GLS

```

# Separate features and target variable
df['EF_GLS']=df['EF']/df['GLS']
df['QRS_GLS']=df['QRS']/df['GLS']
X = df[['EF','QRS','GLS','EF_GLS','QRS_GLS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=10)
svm_model.fit(X_train, y_train)
  
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	12
1	1.00	1.00	1.00	12
accuracy			1.00	24
macro avg	1.00	1.00	1.00	24
weighted avg	1.00	1.00	1.00	24

Cross-validated accuracy scores: [1. 0.91666667 0.875 1. 1.]
 Mean CV accuracy: 0.9583333333333333
 Training Accuracy: 0.9479
 Testing Accuracy: 1.0000
 Training Error: 0.0521
 Testing Error: 0.0000
 No obvious overfitting detected.

Experiment 6: EF_GLS & QRD_GLS, Without GLS feature

```

# Separate features and target variable
df['EF_GLS']=df['EF']/df['GLS']
df['QRS_GLS']=df['QRS']/df['GLS']
X = df[['EF','QRS','EF_GLS','QRS_GLS']] # Features
y = df['HF'] # Target

# Print HF
print(df['HF'].value_counts(normalize=True))

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=10)
svm_model.fit(X_train, y_train)
  
```

```

SVM Accuracy: 1.0
precision    recall  f1-score   support

0           1.00      1.00      1.00        12
1           1.00      1.00      1.00        12

accuracy          1.00      1.00      1.00        24
macro avg          1.00      1.00      1.00        24
weighted avg          1.00      1.00      1.00        24

Cross-validated accuracy scores: [1.          0.91666667 0.875      1.          1.          ]
Mean CV accuracy: 0.9583333333333333
Training Accuracy: 0.9479
Testing Accuracy: 1.0000
Training Error: 0.0521
Testing Error: 0.0000
No obvious overfitting detected.

```

Experiment	CV Accuracy	Training Accuracy	Testing Accuracy	Overfitting
1	0.9583	0.9479	1.0	No
2	0.9583	0.9479	1.0	No
3	0.9583	0.9479	1.0	No
4	0.9583	0.9479	1.0	No
5	0.9583	0.9479	1.0	No
6	0.9583	0.9479	1.0	No

Table 3. Feature Engineering Results.

Conclusion of feature engineering

Table 1 presents the results from all different combinations of feature engineering. All models produced the same results. These results may have different explanations:

1. The first case is that the removed feature, GLS, may have had little or no impact on model performance. The GLS might be unnecessary to the SVM model.
2. Another case is that the features may be strongly correlated. For instance, EF and GLS are both echocardiographic measures and may carry overlapping information. Removing one might not change the model outcome much.
3. The data size is small, and the model is very stable across variations. SVM with C=10 and an RBF kernel is quite robust. If the dataset is small or balanced well, small changes in input features may not affect results.
4. Lastly, the model could be underfitting or reaching performance ceiling. If your dataset is simple and separable, all feature sets might already yield optimal performance.

In conclusion, feature engineering revealed that the GLS feature does not impact model effectiveness. Furthermore, a reduced feature set using only EF and QRS may be preferred for model simplicity and interpretability, without sacrificing predictive power.

Voting Classifier Model – Hard vs Soft Model Evaluation and Comparison

Hard Voting Classifier

The classification ensembles using hard voting with three models: SVM, Random Forest, and XGBoost without GLS feature.

```
# Feature sets
X_all = df[['EF', 'GLS', 'QRS']] # for SVM and RF
X_no_gls = df[['EF', 'QRS']] # for XGBoost

# Scale features
scaler_all = StandardScaler()
X_all_scaled = scaler_all.fit_transform(X_all)

scaler_nogls = StandardScaler()
X_no_gls_scaled = scaler_nogls.fit_transform(X_no_gls)

# Train/test split
X_train_all, X_test_all, y_train, y_test = train_test_split(X_all_scaled, y, test_size=0.2, random_state=42, stratify=y)
X_train_nogls, X_test_nogls, _, _ = train_test_split(X_no_gls_scaled, y, test_size=0.2, random_state=42, stratify=y)

# Define models
svm_model = SVC(kernel='rbf', C=10, probability=False, random_state=42)
rf_model = RandomForestClassifier(n_estimators=100, min_samples_split=10, random_state=42)
xgb_model = XGBClassifier(eval_metric='logloss', colsample_bytree=0.8, learning_rate=0.01,
                          max_depth=3, n_estimators=50, subsample=0.8, random_state=42)

# Train each model
svm_model.fit(X_train_all, y_train)
rf_model.fit(X_train_all, y_train)
xgb_model.fit(X_train_nogls, y_train)

# Custom hard voting
svm_pred = svm_model.predict(X_test_all)
rf_pred = rf_model.predict(X_test_all)
xgb_pred = xgb_model.predict(X_test_nogls)
```

- **Support Vector Classifier with RBF kernel:**
svm_model = SVC (kernel='rbf', C=10, probability=True, random_state=42)
- **Random Forest with 100 trees:**
rf_model = RandomForestClassifier(n_estimators=100, min_samples_split=10, random_state=42),
- **Tuned XGBoost model (without GLS feature):**
xgb_model = XGBClassifier(eval_metric='logloss', colsample_bytree=0.8, learning_rate=0.01, max_depth=3, n_estimators=50, subsample=0.8, random_state=42)
- **Voting classifier:** Combines all 3 classifiers into one ensemble model.
voting='hard': final prediction is based on majority vote (not probabilities).

```
Voting Classifier Accuracy: 0.9583333333333334
Classification Report:
              precision    recall  f1-score   support

     0           1.00        0.92        0.96         12
     1           0.92        1.00        0.96         12

 accuracy          0.96
 macro avg          0.96
 weighted avg       0.96

Confusion Matrix:
[[11  1]
 [ 0 12]]
```

The results indicate that the model classified correctly with 95.83% of the test samples. Although, out of 24 test samples, 23 were predicted correctly.

Results of Class 0 (No Heart Failure):

- Precision: Of the samples predicted as class 0, how many were truly class 0? → 1.0
- Recall: Of all actual class 0 samples, how many were correctly predicted? → 0.92

- F1-score: Harmonic mean of precision and recall → 0.96
- Support: Number of actual samples of class 0 → 12
- The model predicted all class 0 labels very confidently. But it missed 1 real class 0 sample (recall < 1.0).

Results of Class 1 (Heart Failure):

- Precision: Of the samples predicted as class 1, how many were truly class 1? → 0.92
- Recall: Of all actual class 1 samples, how many were correctly predicted? → 1.0
- F1-score: Balanced summary of precision and recall → 0.96
- Support: Number of actual samples of class 1 → 12
- The model captured all true class 1 samples (perfect recall). It made one incorrect class 1 prediction (false positive).

Overall metrics:

- Accuracy: Overall correct predictions out of all test samples → 0.96
- Macro Avg: Average of precision/recall/F1 for both classes, unweighted → 0.96
- Weighted Avg: Average, weighted by class size (both are equal here, so same as macro) → 0.96

Confusion Matrix:

Class	Predicted 0	Predicted 1
Class 0	11	1
Class 1	0	12

Table 4. Hard Voting - Confusion Matrix Results.

- 11 true negatives (TN): actual 0 and predicted 0
- 12 true positives (TP): actual 1 and predicted 1
- 1 false positive (FP): actual 0 predicted as 1
- 0 false negatives (FN): no class 1 missed

Overall, hard voting model is performing very well. It makes one mistake: misclassifies one class 0 as class 1. Precision and recall are high for both classes. Also, the classifier is well-balanced and generalizing nicely.

Soft Voting Classifier

The classification ensembles using soft voting with three models: SVM, Random Forest, and XGBoost without GLS feature.

```
# Prepare different feature sets
X_all = df[['EF', 'GLS', 'QRS']]
X_no_gls = df[['EF', 'QRS']]
y = df['HF'].apply(lambda x: 1 if x in [1, 2] else 0) # Binary classification

# Scale both feature sets
from sklearn.preprocessing import StandardScaler
scaler_all = StandardScaler()
X_all_scaled = scaler_all.fit_transform(X_all)

scaler_no_gls = StandardScaler()
X_no_gls_scaled = scaler_no_gls.fit_transform(X_no_gls)

# Split data (same y)
from sklearn.model_selection import train_test_split
X_train_all, X_test_all, y_train, y_test = train_test_split(
    X_all_scaled, y, test_size=0.2, random_state=42, stratify=y
)
X_train_no_gls, X_test_no_gls, _, _ = train_test_split(
    X_no_gls_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Define models using respective feature sets
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from xgboost import XGBClassifier

svm_model = SVC(kernel='rbf', C=10, probability=True, random_state=42)
rf_model = RandomForestClassifier(n_estimators=100, min_samples_split=10, random_state=42)
xgb_model = XGBClassifier(eval_metric='logloss', colsample_bytree=0.8, learning_rate=0.01,
                          max_depth=3, n_estimators=50, subsample=0.8, random_state=42)

# Fit models separately
svm_model.fit(X_train_all, y_train)
rf_model.fit(X_train_all, y_train)
xgb_model.fit(X_train_no_gls, y_train)
```

- **Support Vector Classifier with RBF kernel:**
svm_model = SVC (kernel='rbf', C=10, probability=True, random_state=42)
- **Random Forest with 100 trees:**
rf_model=RandomForestClassifier(n_estimators=100, min_samples_split=10, random_state=42),
- **Tuned XGBoost model (without GLS feature):**
xgb_model = XGBClassifier(eval_metric='logloss', colsample_bytree=0.8, learning_rate=0.01, max_depth=3, n_estimators=50, subsample=0.8, random_state=42)
- **Voting classifier:** Combines all 3 classifiers into one ensemble model.
voting='soft': Uses predicted probabilities from each model, averages them, and picks the class with the highest average probability. This is often better when classifiers are well-calibrated.

Voting Classifier Accuracy: 0.9583

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.92	0.96	12
1	0.92	1.00	0.96	12
accuracy			0.96	24
macro avg	0.96	0.96	0.96	24
weighted avg	0.96	0.96	0.96	24

Confusion Matrix:

```
[[11  1]
 [ 0 12]]
```

The results indicate that the model classified correctly with 95.83% of the test samples. Although, out of 24 test samples, 23 were predicted correctly.

Results of Class 0 (No Heart Failure):

- Precision → 1.0
- Recall → 0.92
- F1-scope → 0.96
- Support → 12

Results of Class 1 (Heart Failure):

- Precision → 0.92
- Recall → 1.0
- F1-score → 0.96
- Support → 12

Overall metrics:

- Accuracy → 0.96
- Macro Avg → 0.96
- Weighted Avg → 0.96

Confusion Matrix:

Class	Predicted 0	Predicted 1
Class 0	11	1
Class 1	0	12

Table 5. Soft Voting - Confusion Matrix Results.

- 11 true negatives (TN): actual 0 and predicted 0
- 12 true positives (TP): actual 1 and predicted 1
- 1 false positive (FP): actual 0 predicted as 1
- 0 false negatives (FN): no class 1 missed

Overall, Soft Voting ensemble is very accurate at 95.83%. The outcome had only 1 misclassification. Furthermore, there was a balanced precision and recall for both classes. Lastly, excellent F1-scores are achieved (both 0.96), showing the model is both precise and sensitive.

Comparison of Hard and Soft Voting

Both Voting Classifiers, Hard & Soft, performed with 95.83% test accuracy, which shows identical predictive performance.

Comparing the Confusion Matrix of both Voting Classifiers, misclassified 1 instance from class 0 but correctly classified all 12 samples of class 1. Class 0, no heart failure data, interprets that 92% have slightly weaker recall, suggesting 1 false positive result. On the other hand, Class 1, with heart failure data, interprets perfect recall 100%, crucial in clinical applications where detecting heart failure is more critical.

In conclusion, both Voting Classifiers have the same classification results, which show strong correlation among the SVM, Random Forest, and XGBoost models. If we need better theoretical performance and handling probabilistic thresholds or uncertainty, Soft Voting Classifier is identical. On the other hand, Hard Voting Classifier performs with simplicity and speed.

Overall, Soft Voting Classifier provides more flexibility, better theoretical stability and has higher performance in larger datasets.

Conclusion

After we conducted biomedical data analysis with the machine learning technics and possible combinations, Random Forest (RF), Extreme Gradient Boosting (XGBOOST), Extreme Gradient Boosting with Hyperparameter Optimization (XGBOOST+ GridSearchCV), Support Vector Machine (SVM), Voting Classifier and lastly Feature Engineering on the best performing models, we chose the bests performing models which will indicate the best predictive model on the binary classification problem. The results are shown below, on Table 6:

Model	Train Acc.	Test Acc.	Mean CV Acc.	Precision (0/1)	Recall (0/1)	F1-Avg.
RF	0.9896	0.9583	0.9667	1.00/0.92	0.92/1.00	0.96
XGBOOST (No GLS)	0.9792	0.9583	0.9667	1.00/0.92	0.92/1.00	0.96
SVM (RBF C=10)	0.9479	1.00	0.9583	1.00/1.00	1.00/1.00	1.00

Table 6. Best performing models.

By evaluating the above table, we notice that RF and XGBOOST both have high train accuracy but lower test accuracy and have the same mean cross validation accuracy. On the other hand, SVM has lower train accuracy than test accuracy, which has perfect test accuracy (100%), but lower mean cross validation accuracy than the first 2. Overall, the best precision and recall of all tests are generated in SVM model.

In conclusion, the best predictive model turned out to be the SVM RBF kernel model with C=10. No overfitting issue is detected, as train accuracy is lower than tests accuracy. Furthermore, cross validation accuracy is close to training accuracy, which shows that the model performs consistently and generalizes well.